

# Urban Mobility Data Explorer

## **1. Problem Framing and Dataset Analysis**

In this project, we looked at how people move around New York City using public data from the Taxi and Limousine Commission (TLC). We focused on a massive dataset of over 7.6 million Yellow Taxi trips from January 2019. To make sense of all these rides, we mapped them out using a lookup table of 265 taxi zones and a map shapefile. Putting these pieces together gave us a really clear, detailed picture of NYC traffic and travel habits for that month.

### **Data Challenges**

The raw data had several quality issues we had to address:

- Missing or invalid values — corrected or removed
- Negative fares and impossible trip durations — filtered out
- Zero-distance trips and statistical outliers — excluded
- Invalid location IDs and rate codes — removed
- Duplicate records — checked and removed where found

A seven-step cleaning pipeline reduced the dataset from 7,667,792 raw records to 7,280,912 clean records.

### **Assumptions and Unexpected Observation**

Missing congestion surcharge values were treated as zero, unknown location IDs were removed to keep zone-level analysis reliable, tip percentages were only calculated for card payments, since cash tips are not captured in TLC data, and outlier thresholds were set as configurable values rather than hard-coded rules.

One unexpected finding was how clean the dataset already was — the deduplication step removed zero records. This shifted more of our effort toward feature engineering and visualization rather than data repair.

### **Feature Engineering**

To enrich the data for analysis, we created the following additional fields: trip duration, pickup hour, day of week, fare per mile, airport trip flag, average speed, tip percentage, and a fare reconciliation flag.

## **2. System Architecture and Design Decisions**

The system follows a four-layer architecture: data processing, database, API, and frontend. This separation made each part easier to build, test, and debug independently.

View the diagrams below:

**System Architecture Diagram:**

[https://miro.com/app/board/uXjVHG881Ac=/?share\\_link\\_id=58957400262](https://miro.com/app/board/uXjVHG881Ac=/?share_link_id=58957400262)

**Entity Relationship Diagram (ERD):**

[https://miro.com/app/board/uXjVHG9pmKM=/?share\\_link\\_id=151692536485](https://miro.com/app/board/uXjVHG9pmKM=/?share_link_id=151692536485)

### **Technology Choices**

- Python: data cleaning, validation, and transformation
- PostgreSQL (Aiven): stores cleaned trip data and supports analytical queries
- Flask: API layer connecting the database to the frontend
- HTML, CSS, JavaScript: dashboard interface built without a frontend framework

# Urban Mobility Data Explorer

- Leaflet: geographic visualization of taxi zones on a map

## Schema Design

The database is centered around a trips table. Supporting tables for zones, vendors, payment types, and rate codes reduce data duplication and keep the schema consistent.

## Design Trade-offs

- Storage limits on Aiven's free tier meant we loaded a representative subset of the 7.2 million cleaned records rather than the full dataset.
- We chose plain JavaScript over a frontend framework to reduce complexity, though this required more manual coding.

## 3. Algorithmic Logic and Data Structures

One core challenge was identifying the top pickup zones from millions of trip records. Sorting every zone by count would work but becomes slow as the number of zones grows. Instead, we implemented a custom Top-K Zones algorithm using a hash map and a manually built MinHeap — with no reliance on built-in sorting or counting libraries.

## Approach

First, trip counts per zone are stored in a hash map. Then, a MinHeap of size  $k$  keeps track of only the top candidates. Each new zone replaces the current minimum only if its count is higher. This avoids sorting all zones entirely.

## Pseudocode

Access the pseudocode here:

[https://docs.google.com/document/d/1HLMBRJbIVg9Isa9f\\_KXKD7eSuW8TNcrUFUBPm0-n\\_XA/edit?usp=sharing](https://docs.google.com/document/d/1HLMBRJbIVg9Isa9f_KXKD7eSuW8TNcrUFUBPm0-n_XA/edit?usp=sharing)

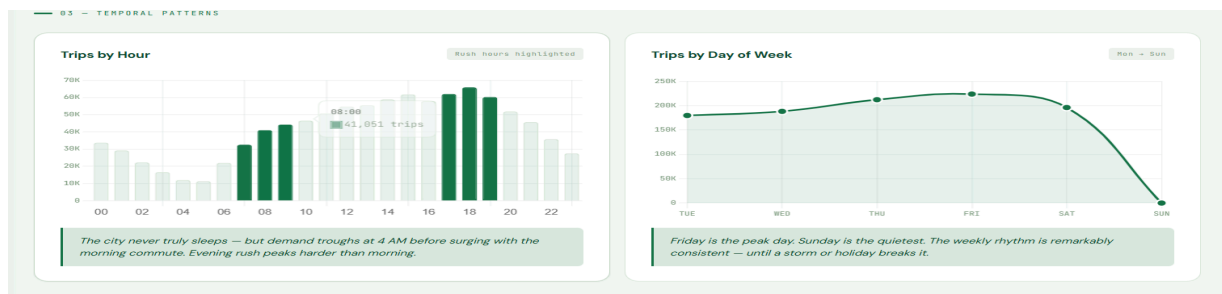
## Complexity Analysis

- Time Complexity:  $O(n + m \log k)$
- Space Complexity:  $O(m + k)$

Where  $n$  = total trips,  $m$  = unique zones,  $k = 10$ . This is significantly faster than sorting all zones, which would cost  $O(m \log m)$ .

## 4. Insights and Interpretation

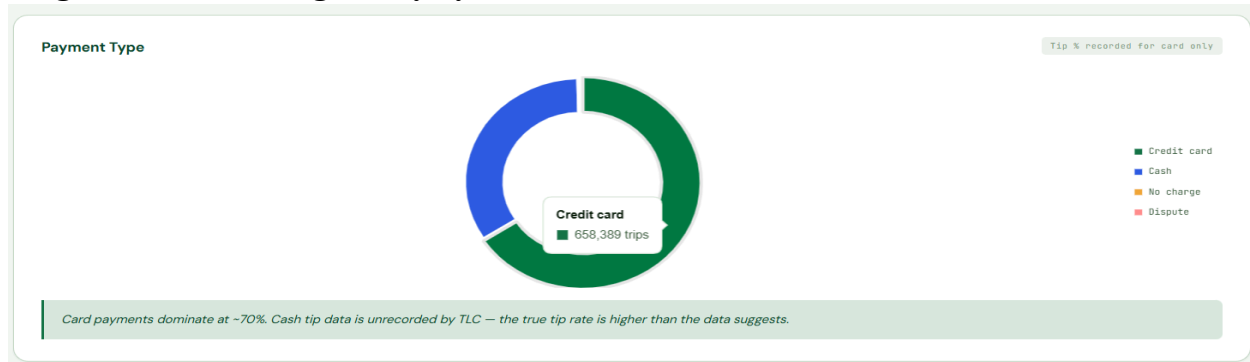
### Insight 1: Taxi Demand Changes by Time



# Urban Mobility Data Explorer

We grouped trips by pickup hour and day of week. Evening hours consistently showed the highest demand, with Friday recording the most trips overall. This confirms that taxi usage closely follows work and social activity patterns.

## Insight 2: Most Passengers Pay by Card



Grouping trips by payment type showed that credit card payments outnumber cash transactions by a wide margin. This reflects a broader shift toward digital payments in urban transit.

## Insight 3: Longer Trips Cost Less Per Mile



We compared trip distance against fare per mile. The cost per mile decreases as the trip gets longer, which makes sense — fixed charges like base fares and surcharges have a bigger relative impact on short rides than long ones.

## 5. Reflection and Future Work

Two main challenges shaped how we worked. First, Aiven's free storage tier could not hold the full 7.2 million records, which meant we had to work with a subset. Second, coordinating across four distinct areas — data cleaning, database design, backend development, and frontend — required consistent communication and careful integration.

If we continued this project, the next steps would be: loading the full cleaned dataset on larger cloud infrastructure, adding demand prediction features based on time and zone, and improving the dashboard with real-time data refresh. These additions would take the project closer to a production-ready mobility analysis tool.